

Stonks

Interview Study Guide

100 difficult, project-specific questions with concise interview-ready answers

Prepared from the current Stonks codebase: Next.js, MySQL, JWT authentication, market-data integrations, FastAPI ML services, and Chroma-powered AI chat.

Architecture and Next.js

1. Why choose Next.js full stack over separate frontend and backend?

Answer: It reduces deployment and integration complexity, lets UI and BFF routes share types and auth conventions, and is ideal for this product stage. A separate backend becomes worthwhile when independent scaling, long-running work, multiple clients, or stronger service boundaries justify it.

2. What are the benefits and limits of App Router route handlers?

Answer: They colocate API behavior with the web app, support streaming and server rendering, and simplify deployment. They are less suitable for persistent connections, CPU-heavy jobs, and independently scaled services.

3. Describe the flow when an authenticated user opens /dashboard/portfolio.

Answer: The proxy verifies the access token, or permits a valid refresh token path. The page loads and calls protected portfolio APIs; the route re-verifies the access JWT, loads wallet, holdings, and transactions from MySQL, then renders the response.

4. What gap exists if only dashboard pages are protected in proxy.ts?

Answer: API endpoints remain directly callable. Every sensitive route must independently authenticate and authorize; middleware is defense in depth, not the complete security boundary.

5. Why does proxy.ts allow requests when secrets are absent, and why is that risky?

Answer: It avoids breaking local setup before environment configuration. In production it could silently disable dashboard protection, so production should fail closed with startup validation.

6. What risks come from mixing JS and TS API routes?

Answer: Contracts become easier to drift, refactors lose compile-time guarantees, and runtime shape errors rise. Shared schemas and a gradual migration to TypeScript reduce this.

7. Which code should be server versus client components?

Answer: Data-sensitive or initial data-fetching UI should be server-side. Interactive charts, forms, theme toggles, and localStorage wishlist need client components. The goal is minimum client JavaScript and no secret leakage.

8. How avoid duplicate dashboard market requests?

Answer: Fetch at a page or server boundary, cache/dedupe by symbol and TTL, then pass data down. Client libraries such as SWR can deduplicate and revalidate.

9. How would you scale to 10,000 concurrent users?

Answer: Use CDN caching for static assets, distributed cache and request coalescing for quotes, managed MySQL with pooling, rate limits, queues for slow jobs, separately autoscaled ML, and load testing with observability.

10. Why is global in-memory cache unreliable on Vercel?

Answer: Instances are ephemeral and isolated. Cold starts and horizontal scaling mean each instance has a different cache, so use Redis/KV or another shared store for correctness.

Authentication and Security

11. Walk through signup end to end.

Answer: The route validates input, ensures schema availability, checks email uniqueness, hashes the password with bcrypt, inserts the user, creates signed access and refresh JWTs, stores a hash of the refresh token, and returns secure cookies.

12. Why bcrypt cost 12?

Answer: It intentionally makes hashing expensive enough to slow offline password guessing. The trade-off is login/signup latency and CPU; it must be benchmarked under expected load.

13. Why hash refresh tokens in MySQL?

Answer: A database leak should not expose bearer credentials that can immediately authenticate users. The supplied token is hashed and compared instead.

14. What happens with one refresh-token hash and two devices?

Answer: The newer login overwrites the stored hash, effectively revoking the other device on its next refresh. Multi-device support needs a token-session table.

15. How redesign refresh tokens for multi-device use?

Answer: Store one row per session with token-family ID, hashed token, device metadata, expiry, revocation timestamp, and rotation lineage. Rotate on each refresh and revoke the family when reuse is detected.

16. Why use short-lived access tokens?

Answer: They limit damage from theft while avoiding a database lookup on every request. Refresh tokens provide a controlled way to extend sessions.

17. What is wrong if proxy accepts a refresh token but does not rotate access?

Answer: The page request can proceed while downstream API calls still lack a valid access token. A refresh endpoint or middleware response must mint/set a new access token safely.

18. Explain HttpOnly, Secure, SameSite, path and maxAge.

Answer: HttpOnly blocks JavaScript access; Secure limits cookies to HTTPS; SameSite reduces cross-site sending; path scopes where cookies are sent; maxAge controls persistence. These should be deliberate for auth cookies.

19. What CSRF protection is needed?

Answer: Cookie auth is automatically attached by browsers, so state-changing routes need SameSite policy plus origin checks and a CSRF token or double-submit strategy where cross-site use is possible.

20. When use 401, 403, 409 and 500?

Answer: 401 is absent/invalid authentication; 403 is authenticated but not allowed; 409 is a conflict such as duplicate email; 500 is an unexpected server fault. Validation errors are normally 400.

21. Why does JWT validity not guarantee immediate logout?

Answer: A signed access token remains valid until expiry. Use short expiry and session/token-version checks or deny lists for urgent revocation.

22. How defend sign-in endpoints?

Answer: Rate-limit by IP and account, add progressive delays or CAPTCHA, monitor anomalies, use generic invalid-credential responses, and enforce strong passwords and MFA for sensitive accounts.

23. Why have Firebase client/Admin alongside custom JWT?

Answer: Firebase client supports browser auth and Admin verifies server-side Firebase tokens. Mixing it with custom JWTs can create two identities and inconsistent authorization; choose one authority or explicitly bridge them.

Database and Transactions

24. Why use transaction plus SELECT FOR UPDATE for a trade?

Answer: The transaction groups wallet, holdings, and ledger changes atomically. FOR UPDATE locks the relevant rows so concurrent trades cannot both act on the same stale balance or quantity.

25. What race occurs without row locks?

Answer: Two BUYs can both observe Rs.1000, each spend Rs.800, and both commit, producing an impossible negative or overstated balance. Similarly two SELLs can oversell one holding.

26. Why must all trade writes be atomic?

Answer: A wallet debit without a holding, or a holding change without a transaction ledger, corrupts financial state. Commit only when every invariant has been preserved.

27. Why delete a zero-quantity holding?

Answer: It keeps active positions simple and avoids zero rows in views. Retaining it preserves position history but requires filtering; the transaction table remains the audit record either way.

28. Explain weighted average cost basis.

Answer: After a buy, new average = (old quantity x old average + bought quantity x buy price) / new quantity. It gives a single blended cost basis for the remaining position.

29. Why realized P&L; only on SELL?

Answer: A buy establishes cost but does not close economic exposure. Selling crystallizes gain or loss relative to cost basis; unsold changes are unrealized.

30. What is a weakness of weighted average versus FIFO?

Answer: It cannot identify the precise tax lot sold. FIFO, LIFO, or specific identification may be legally or financially required and produce different realized P&L;

31. Why prefer DECIMAL to JavaScript Number for money?

Answer: Binary floating point cannot exactly represent many decimal currencies. DECIMAL preserves exact fixed precision in storage and calculations; application code should use decimal arithmetic or integer minor units.

32. What precision issue remains in roundMoney(quantity * price)?

Answer: The multiplication can already contain a binary floating error before rounding, and repeated rounding can accumulate drift. Use decimal math and a documented rounding mode.

33. How make trade POST idempotent?

Answer: Require a unique client order ID/idempotency key, store its payload hash and completed response inside the transaction, and return that response for retries instead of executing again.

34. Why is dynamic schema creation in request paths risky?

Answer: It adds latency, needs DDL permissions, can race during deploys, and hides migration history. Use versioned migrations in CI/CD.

35. What indexes are needed?

Answer: Unique users(email), unique wallets(user_id), unique holdings(user_id,sym), and transactions(user_id,created_at,id) for account history. Add only indexes that match actual query patterns.

36. How model splits, dividends, fees, fills and cancels?

Answer: Use an immutable orders/executions ledger with event types. Corporate actions adjust lots; fees are explicit cash events; order state is separate from fills; cancellations never erase history.

37. How implement transaction pagination?

Answer: Use a stable cursor such as (created_at,id), query rows older than that tuple, and return nextCursor. Offset pagination becomes slow and unstable while new rows arrive.

Market Data and Portfolio Logic

38. How establish a single quote source of truth across providers?

Answer: Create a normalized quote service with provider priority, timestamp, currency, exchange, session and provenance. All routes consume that canonical format, not provider-specific shapes.

39. How expose quote freshness?

Answer: Return source, asOf timestamp, market session, delay and fallback/stale flags. The UI must label estimates and prevent them being mistaken for executable live prices.

40. What can differ between providers?

Answer: Currency, exchange listing, split adjustment, timezone, regular versus extended session, and timestamp. Mixing them can create invalid comparisons and P&L;

41. How handle rate limits gracefully?

Answer: Cache results, deduplicate concurrent fetches, back off after 429, use circuit breakers, display last-known data with age, and avoid retry storms.

42. What cache design would you use for quotes?

Answer: Key by normalized symbol and session, use a short TTL for live quotes, stale-while-revalidate to serve recent data, request coalescing on miss, and include provider timestamp rather than assuming freshness.

43. Why use timeout and AbortController for API calls?

Answer: A hung provider should not occupy a serverless invocation until it fails globally. Bounded work keeps the page responsive and allows controlled fallback.

44. Why is chat calling its own HTTP market API inefficient?

Answer: It adds routing, serialization, auth/caching ambiguity and an extra failure hop. Extract the market service into a shared server module and call it directly.

45. How avoid cache stampede?

Answer: Use single-flight locks/request coalescing, jittered TTLs, stale responses while refreshing, and per-key distributed locks when needed.

46. How handle non-trading periods?

Answer: Return market status and last close separately from intraday quote. Do not present stale close as live; account for exchange calendar and timezone.

47. Why not trust client-sent trade price?

Answer: A client can alter it. For a simulator, server must get a validated server-side quote or clearly allow educational custom pricing; real execution requires broker-confirmed fills.

48. What are invested value, market value, realized, unrealized P&L; and equity?

Answer: Invested is remaining cost basis; market value is quantity times current price; realized P&L; is closed trade result; unrealized P&L; is current value minus cost; equity is cash plus market value.

49. How measure portfolio return with cash flows?

Answer: Use time-weighted return to isolate manager performance or money-weighted return/IRR to reflect investor cash timing. A simple ending-minus-starting balance is misleading.

50. How measure diversification?

Answer: Examine position weights, sector/geographic/factor exposures, correlation, concentration measures such as HHI, and liquidity - not merely number of holdings.

51. What risks come from portfolio ratings?

Answer: Users may treat model output as personalized investment advice. Add suitability boundaries, uncertainty/explanations, data-quality warnings, human/legal review, and avoid imperative personalized claims.

ML Service and Model Engineering

52. Why separate FastAPI ML from Next.js?

Answer: It isolates Python dependencies and model memory/CPU, allows independent deployment/scaling/versioning, and avoids unsuitable Node/serverless inference paths.

53. Why does EOD prediction support only AAPL and TSLA?

Answer: Those are the available trained artifacts and preprocessing pipelines. The API should reject unsupported tickers clearly, as it does, rather than imply universal coverage.

54. Differentiate the project ML features.

Answer: EOD predicts a future price; buy-signal classifies a trading action; portfolio rater scores composition; news sentiment labels text; chatbot generates language with retrieved context. They have distinct data, risks, and metrics.

55. Why is pseudo-confidence misleading?

Answer: It is derived from return magnitude, not calibrated probability. A user may interpret it as model certainty despite no calibration evaluation.

56. How define buy/sell labels without look-ahead bias?

Answer: At time t use only information available at t , set a future-horizon return threshold for the label, and ensure all features are timestamp-aligned and delayed where necessary.

57. Give examples of leakage.

Answer: Using future close in an indicator, applying scaler fit on all data, using revised fundamentals unavailable then, or randomly mixing later samples into training.

58. Why is random splitting wrong for market time series?

Answer: It puts future regimes in training while evaluating earlier samples, exaggerating performance. Training must precede validation in time.

59. Explain walk-forward validation.

Answer: Train on an initial historical period, validate on the next slice, advance the window, and repeat. It simulates repeated real deployment across regimes.

60. Why can accuracy be useless for trading?

Answer: A model can predict common small moves accurately but lose on rare large moves, fees, slippage and bad position sizing. Profitability and risk metrics matter.

61. What metrics matter?

Answer: Regression: MAE/RMSE and directional error; classification: precision, recall, ROC/PR and calibration; strategy: net return, drawdown, Sharpe/Sortino, turnover, hit rate and capacity.

62. How include trading frictions in backtests?

Answer: Apply bid-ask spread, commissions, slippage, delays, liquidity limits, borrow costs and market impact at realistic execution times. Test sensitivity rather than one optimistic assumption.

63. What drift should be monitored?

Answer: Feature drift, label/performance drift, regime change, provider changes and latency/failure drift. Compare live distributions and delayed realized outcomes with retraining governance.

64. When fallback heuristic versus hard failure?

Answer: Fallback only for a clearly labelled, safe, lower-confidence experience; hard-fail when output could be mistaken for a reliable decision or inputs/model version are invalid.

65. What does PY_AI_SERVICE_URL solve?

Answer: Next.js forwards inference to an independently deployed FastAPI service, avoiding Vercel limitations around local Python child processes and heavy model dependencies.

66. How deploy the ML service?

Answer: Containerize with pinned artifacts, health/readiness endpoints, autoscale based on latency/queue/CPU, version API and models, secure service-to-service auth, and canary deployments with rollback.

67. How protect ML endpoints?

Answer: Authenticate callers, rate-limit and cap payload sizes, enforce timeouts/concurrency limits, validate tickers/text, queue expensive work, and monitor abuse and cost.

AI, RAG, Reliability and Testing

68. Describe the chat lifecycle.

Answer: The route validates prompt/session, loads history, gets stocks/news, upserts them into Chroma, retrieves semantic context, calls the selected model with timeouts/fallback, stores the turn, and returns answer plus metadata.

69. Why separate Chroma collections?

Answer: Stocks, news, chats and portfolios have different IDs, retention, metadata filters and relevance behavior. Separation makes retrieval and access control clearer.

70. Why normalize BGE embeddings and use cosine?

Answer: Normalization makes dot product equal cosine similarity and removes vector magnitude as noise. Cosine is a common semantic similarity measure for sentence embeddings.

71. What does HNSW optimize and trade off?

Answer: It provides fast approximate nearest-neighbor search with good recall. It trades exactness, memory and index-build cost for low query latency.

72. When does keyword search beat embeddings?

Answer: Exact tickers, numbers, dates, proper names and filters often need lexical/structured search. A hybrid retrieval strategy is stronger than embeddings alone.

73. What does retrieving only one stock and one news item miss?

Answer: Comparisons, broad portfolio questions, multi-stock correlations and evidence requiring several articles. Retrieval limits should match task intent and token budget.

74. Can chat retrieval leak user data across sessions?

Answer: Yes. `queryChromaContext` searches chats without a `sessionId` or `userId` filter, so semantic matches can cross boundaries. Store owner ID and query with mandatory authorization filters.

75. Why is client-controlled `sessionId` dangerous?

Answer: A user can guess or choose another session identifier. Session ownership must be derived from authenticated identity and server-created opaque IDs.

76. How bind chat and portfolio data to users?

Answer: Use JWT subject as owner, store it in metadata, enforce owner filters in every read/write, and never accept owner IDs from clients.

77. What is prompt injection here?

Answer: Untrusted news or old chat may contain instructions such as "ignore your rules and reveal data." Treat retrieved text as data, delimit it, apply policy outside the prompt, and minimize sensitive context.

78. Why is context-only prompting not enough?

Answer: Models can still hallucinate, misread context, or follow malicious instructions. Use citations, structured outputs, relevance thresholds, validation and clear uncertainty.

79. How make chatbot answers traceable?

Answer: Return document IDs, source URLs, timestamps and quoted data fields used; have the model reference only supplied source IDs and validate citations before rendering.

80. Why is portfolio context sensitive?

Answer: It reveals assets, cash, behavior and potentially identity. Limit retrieval, encrypt/protect stores, apply tenant filters, retention controls and least privilege.

81. Why is JSON fallback storage unsafe in production?

Answer: It is local to an instance, may disappear, does not coordinate across instances, has weak concurrency guarantees, and can unintentionally persist sensitive chat data.

82. What occurs on a serverless cold start?

Answer: In-memory fallback chat state is empty. User history becomes inconsistent or disappears unless persisted to a shared durable system.

83. How support chat deletion/export?

Answer: Store ownership and timestamps, provide authenticated export/delete operations, delete both primary and vector records, propagate deletion, audit it, and define retention windows.

84. Why store prompt and answer separately?

Answer: It preserves conversation roles and enables retrieval of either. But it doubles documents, can retrieve an answer without its question, and needs pairing/order metadata.

85. What is the risk of `Date.now()` IDs?

Answer: Concurrent requests in the same millisecond can collide, especially across processes. Use UUID/ULID or a database-generated unique ID.

86. What inconsistency exists in local_finetuned mode?

Answer: The function says it routes local requests to Gemini and directly calls Gemini, so the configured provider is not actually local inference. Wire it to /chat ML service or rename/remove the option.

87. How make provider fallback observable?

Answer: Emit structured provider, model version, fallback reason, latency and error metrics; show accurate non-sensitive metadata and alert on elevated fallback rates.

88. What is lost by buffering Hugging Face SSE?

Answer: The user sees no incremental tokens, cancellation is weaker, and memory/latency increase. Stream validated events through a server response instead.

89. How control LLM spend?

Answer: Authenticate, per-user quotas and rate limits, input/output token caps, caching where safe, request budgets, model routing, abuse monitoring and hard circuit breakers.

90. Where use retries, timeouts, circuits and fallbacks?

Answer: Timeout all external calls; retry idempotent transient reads with jitter; circuit-break persistently failing providers; use safe stale/fallback data. Never blindly retry a trade POST.

91. Why is retrying POST trade dangerous?

Answer: A network timeout may happen after commit. Retrying can duplicate a buy/sell unless idempotency keys guarantee one logical order.

92. What logs are safe and useful?

Answer: Use correlation ID, endpoint, authenticated user pseudonym/ID, provider, duration, status and sanitized error code. Never log passwords, JWTs, API keys, raw sensitive prompts or full financial data by default.

93. What dashboards and alerts matter?

Answer: Latency/error/rate-limit dashboards by route/provider, quote freshness, DB pool/lock waits, ML latency, fallback rate, auth failures, abnormal trade patterns and cost. Alert on SLO breaches and security anomalies.

94. How divide testing layers?

Answer: Unit-test math/validation; integration-test DB transactions and schema; API-test auth/contracts; E2E-test critical user paths; contract-test provider/ML payloads. Keep a small deterministic fixture set.

95. Give five critical trade tests.

Answer: Insufficient cash buy rejected; oversell rejected; weighted average after multiple buys correct; wallet/holding/ledger atomically roll back on failure; concurrent requests cannot overspend or oversell.

96. How test trade races?

Answer: Run concurrent POSTs against a real transactional test MySQL database with barriers so both attempts contend on the same rows; then assert one valid final state and no broken invariants.

97. How mock external systems well?

Answer: Use protocol-accurate fixtures, simulated 429/timeouts/malformed responses, contract tests against sandbox providers, deterministic clocks and a disposable real DB/vector service for integration tests.

98. What load tests run before launch?

Answer: Quote burst and cache-miss load, sign-in abuse patterns, concurrent same-wallet trades, chat fan-out with upstream slowness, ML saturation, and soak tests for connection leaks.

99. How diagnose slow chat?

Answer: Trace one request across quote/news fetch, embedding, Chroma, model inference and storage using correlation IDs, timings and distributed traces; compare p50/p95/p99 by dependency.

100. Which secrets must stay server-only?

Answer: Database credentials, JWT secrets, Firebase Admin private key, Gemini/Hugging Face/provider keys and internal service credentials. Only intentionally public Firebase client config may use NEXT_PUBLIC_. Scan bundles/logs and enforce secret checks in CI.

101. Why keep .env.example but no secrets?

Answer: It documents required configuration and supports reproducible deploys, but committing usable keys exposes systems permanently through Git history.

102. What is the incident plan for wrong recommendations?

Answer: Immediately disable or gate the feature, preserve audit evidence, notify affected users as required, investigate data/model/provider changes, correct the issue, validate with staged rollout, and document the postmortem.